National University of Singapore
School of Computing
CS3244: Machine Learning
Solution to Tutorial 8

Decision Trees and Ensemble Methods

Readings: *CIML, Chapter 1 (Decision Trees) & Chapter 13 (Ensemble Methods).*

1. **Decision Trees.**

   The loans department of DBN (Development Bank of NUS) has the following past loan processing records each containing an applicant's income, credit history, debt, and the final approval decision.

Income	Credit History	Debt	Decision
0 – 5K	Bad	Low	Reject
0 – 5K	Good	Low	Approve
0 – 5K	Unknown	High	Reject
0 – 5K	Unknown	Low	Approve
0 – 5K	Unknown	Low	Approve
0 – 5K	Unknown	Low	Reject
5 – 10K	Bad	High	Reject
5 – 10K	Good	High	Approve
5 – 10K	Unknown	High	Approve
5 – 10K	Unknown	Low	Approve
Over 10K	Bad	Low	Reject
Over 10K	Good	Low	Approve

   (a) **AdaBoost + Decision Stumps.** Construct the AdaBoost series of decision stumps given the same set of examples. How many iterations does AdaBoost take before terminating?

   We will only show 2 rounds of boosting as this will be sufficient to get the idea behind AdaBoost.

   In the first round, AdaBoost will choose the same decision stump as the complete decision tree, choosing Credit History as the attribute for the decision as all of the examples have the same weight $\frac{1}{12}$. *Note that the Credit History == "Unknown" defaults to "Approve" since* $\frac{4}{6}$ *of the examples indicate "Approve". The classifier gets* $\frac{10}{12}$ *correct in total, missing the 3rd and 6th examples in the original table. Hence,* $\epsilon_1 = \frac{1}{6}$ *and* α_1 *(the weight for this first classifier)* $= \frac{1}{2}ln(\frac{1-\frac{1}{6}}{\frac{1}{6}}) = \frac{1}{2}ln5 \approx .80$.

   For the second round, we calculate $exp(-\alpha_1 y_n g_1(\mathbf{x}_n))$, *where we take* $y_n g_1(\mathbf{x}_n) = 1$ *for examples that are correct and* -1 *for examples that are incorrect. Weights for the second round are thus* $\mathcal{D}_2 = \frac{1 \cdot e^{-.8}}{Z_1} \approx \frac{0.45}{Z_1} = 0.05$ *for correct examples,* $\approx \frac{2.22}{Z_1} = 0.25$ *for*

incorrect examples (before and after $Z_1 = 8.94$ normalization over all 12 points).

In the second round, we must redo the information gain calculation in the face of the new weights for the examples. At the root now, we have: $H(\mathcal{D}) \approx H(0.65, 0.35) \approx 0.93$

$$\begin{aligned}
R_{Income} &= 0.7 \times H(0.21, 0.79) + 0.2 \times H(0.25, 0.75) + 0.1 \times H(0.5, 0.5) \\
&= 0.7 \times 0.74 + 0.2 \times 0.81 + 0.1 \times 1.0 = 0.52 + 0.16 + 0.08 = 0.76 \\
IG_{Income} &= 0.93 - 0.76 = 0.17 \\
R_{CreditHistory} &= 0.15 \times H(1, 0) + 0.15 \times H(0, 1) + 0.7 \times H(0.44, 0.55) \\
&= 0 + 0 + 0.7 \times 0.99 = 0.69 \\
IG_{CreditHistory} &= 0.93 - 0.69 = 0.24 \\
R_{Debt} &= 0.6 \times H(.58, .41) + 0.4 \times H(.75, .25) \\
&= 0.6 \times 0.98 + 0.4 \times 0.81 = 0.59 + 0.32 = 0.91 \\
IG_{Debt} &= 0.93 - 0.91 = 0.02
\end{aligned}$$

where in the above, "R" denotes remainder, and "IG" denotes information gain.

With the changes in weighting, we see that the information gain of Credit History has decreased a bit, and that both Debt and Income have increased. However, with one step, the weights have not sufficiently changed to change the root (decision stump)'s choice of attribute to split on, and we can see that the classifier still picks Credit History, and gets the same misclassification rate $\epsilon_2 = \frac{1}{6}$.

This would be further perturbed in later rounds of the boosting algorithm and would eventually select the Income attribute to split on, choosing to "Reject" cases with "0 – 5K" income as that would classify the two incorrect examples (#3 and #6) correctly.

In reality, we need to define a termination criterion that will allow the algorithm to terminate. As with other iterative algorithms, we can see that the classification of the points has not changed over two iterations in this example, yet the weights on the points are significantly changing. As such it is often useful **not** *to set the termination criterion that is only based on the stability of the classification (as the weights can change over several iterations that will eventually result in a change in classification).*

(b) **Random Forest.** Construct a random forest of size 3, where each of the three decision trees is fully grown from two of the three attributes, again based on the same set of examples: {Income, Credit History}, {Credit History, Debt} and {Debt, Income}.

Based on the earlier answers for the decision tree, we know that Credit History will be the root node as it has a better information gain. This leads to pure "Bad" and "Good" leaves, leading to only using the secondary attribute for the remaining six cases of "Unknown".

The {Income, Credit History} tree is identical to the one for the full decision tree, except that the bottom Debt node is left off and replaced by the mode of the examples that fulfill the tests (Credit History == "Unknown" ∩ Income == "0 – 5K"). This is a tie, so the parent node's test of just Credit History == "Unknown" is used to obtain "Approve".

For the {Credit History, Debt} tree, the Credit History == "Unknown" takes a Debt test, resulting "Approve" for "Low" values (by the mode of examples), and also "Approve" for "High" values (by the mode of the parent, since the mode of the examples is tied).

For the {Debt, Income} tree, we can recall that the information gain of the previous decision tree showed that Income was better than Debt. All three subcases of Income are not pure and each needs a test for Debt. In the "0 – 5K" Income, Debt == "Low" yields "Approve" by the mode of the examples, and yields "Reject" for "High" Debt cases. In the "5 – 10K" Income, Debt == "High" yields "Approve" by the mode of the examples, and yields "Approve" by purity when Debt is "Low". In the "Over 10K" Income, there are no examples of "High", so it defaults to "Approve". Similarly, since there is a tied outcome for Debt == "Low", we default to the mode of all examples, hence "Approve".

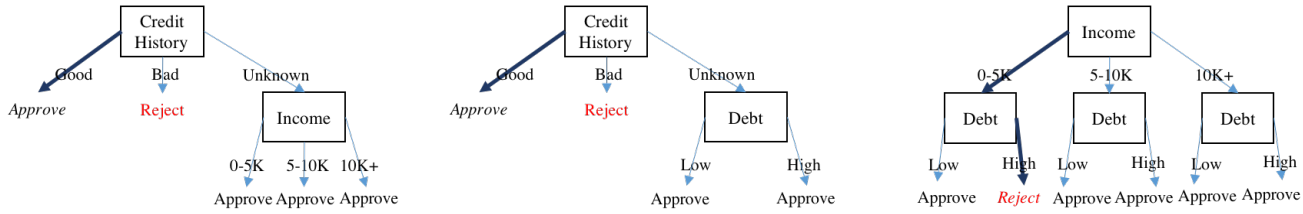

Figure 1: Random Forest of size 3 in Part (c). The dark edges and italicized labels correspond to the decision for each tree for the individual described in Part (d).

2. **Uniform Blending.** One of the simplest ensemble methods is Uniform Blending. Given a set of hypothesis: $h_1, h_2, h_3, ..., h_T$, Uniform blending makes predictions simply by mixing the predictions given by $h_1, h_2, h_3, ..., h_T$ uniformly. Concretely, for binary classification, uniform blending predicts by:

$$H(x) = \text{sign}(\sum_{t=1}^{T} h_t(x)), \tag{1}$$

and for regression, uniform blending predicts by:

$$H(x) = \frac{1}{T}\sum_{t=1}^{T} h_t(x) \tag{2}$$

Taking regression as an example, show that the performance (measured by out-of-sample error) of uniform blending is no worse than the average performance over $h_1, h_2, h_3, ..., h_T$ i.e.

$$\frac{1}{T}\sum_{t=1}^{T} J_{\text{test}}(h_t(x)) \geq J_{\text{test}}(H(x)) \tag{3}$$

Assume we evaluate the testing error by mean square error.

Hint: Start by calculating the average error over $h_1, h_2, h_3, ..., h_T$ for one fixed data point x.

Proving equation 3 can give us an intuition on why ensembling can help to reduce the out-of-sample error.

Let's first see the case for only one fixed data point x. Suppose the ground truth function is $f(x)$. The average error over $h_1, h_2, h_3, ..., h_T$ for point x is

$$\frac{1}{T}\sum_{t=1}^{T}[h_t(x) - f(x)]^2 = \frac{1}{T}\sum_{t=1}^{T}\left[h_t(x)^2 - 2h_t(x)f(x) + f(x)^2\right] \tag{4}$$

$$= \frac{1}{T}\sum_{t=1}^{T}h_t(x)^2 - \frac{2f(x)}{T}\sum_{t=1}^{T}h_t(x) + \frac{1}{T}\sum_{t=1}^{T}f(x)^2 \tag{5}$$

$$= \frac{1}{T}\sum_{t=1}^{T}h_t(x)^2 - 2f(x)H(x) + \frac{1}{T}\sum_{t=1}^{T}f(x)^2 \qquad H(x) = \tfrac{1}{T}\textstyle\sum_{t=1}^{T}h_t(x) \tag{6}$$

$$= \frac{1}{T}\sum_{t=1}^{T}h_t(x)^2 - 2f(x)H(x) + f(x)^2 \qquad f(x) \text{ does not depend on } t \tag{7}$$

$$= \frac{1}{T}\sum_{t=1}^{T}h_t(x)^2 - H(x)^2 + H(x)^2 - 2f(x)H(x) + f(x)^2 \tag{8}$$

$$= \frac{1}{T}\sum_{t=1}^{T}h_t(x)^2 - H(x)^2 + [H(x) - f(x)]^2 \tag{9}$$

$$= \frac{1}{T}\sum_{t=1}^{T}h_t(x)^2 - 2H(x)^2 + H(x)^2 + [H(x) - f(x)]^2 \tag{10}$$

$$= \frac{1}{T}\sum_{t=1}^{T}h_t(x)^2 - \frac{1}{T}\sum_{t=1}^{T}2h_t(x)H(x) + \frac{1}{T}\sum_{t=1}^{T}H(x)^2 + [H(x) - f(x)]^2 \tag{11}$$

$$= \frac{1}{T}\sum_{t=1}^{T}\left[h_t(x)^2 - 2h_t(x)H(x) + H(x)^2\right] + [H(x) - f(x)]^2 \tag{12}$$

$$= \frac{1}{T}\sum_{t=1}^{T}[h_t(x) - H(x)]^2 + [H(x) - f(x)]^2 \tag{13}$$

To compute the out-of-sample error, we only need to compute the expected error over all x,

hence, the average out-of-sample error over $g_1, g_2, g_3, ..., g_T$ *is*

$$\frac{1}{T}\sum_{t=1}^{T} J_{test}(h_t(x)) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{E}_x\left\{[h_t(x) - f(x)]^2\right\} \tag{14}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T}[h_t(x) - f(x)]^2\right\} \tag{15}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T}[h_t(x) - H(x)]^2 + [H(x) - f(x)]^2\right\} \tag{16}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T}[h_t(x) - H(x)]^2\right\} + \mathbb{E}_x\left\{[H(x) - f(x)]^2\right\} \tag{17}$$

$$= \mathbb{E}_x\left\{\frac{1}{T}\sum_{t=1}^{T}[h_t(x) - H(x)]^2\right\} + J_{test}(H(x)) \tag{18}$$

$$\geq J_{test}(H(x)) \tag{19}$$

From the above deduction we can see, the average out-of-sample error over $h_1, h_2, h_3, ..., h_T$ *is greater than or equal to the out-of-sample error of* $H(x)$*. Hence, the performance (measured by out-of-sample error) of uniform blending is no worse than the average performance over* $h_1, h_2, h_3, ..., h_T$*.*